JAVA PROGRAMMING ASSIGNMENT

Paper 1

Q1. What are the features of OOPs? Explain.

Object-Oriented Programming (OOP) is a programming approach in which programs are organized around classes and objects rather than only functions. Java uses OOP concepts to make software modular, reusable, and easier to maintain.

The main features of OOP are:

1. Class

A class is a blueprint or template used to create objects. It defines the data members and methods that its objects can use.

```
class Student {
int rollNo;

    void display() {
System.out.println(rollNo);
    }
}
```

2. Object

An object is an instance of a class. It has state, represented by data, and behaviour, represented by methods.

```
Student s = new Student();
```

3. Encapsulation

Encapsulation means combining data and methods into a single unit and controlling access to the data. In Java, access modifiers such as **private** help protect class data.

4. Inheritance

Inheritance allows one class to acquire accessible fields and methods from another class. It reduces code duplication and supports code reuse. `class Dog extends Animal { }`

5. Polymorphism

Polymorphism means 'many forms'. The same method name can behave differently through method overloading or method overriding.

6. Abstraction

Abstraction means hiding internal implementation details and showing only essential features. Java supports abstraction through abstract classes and interfaces.

Conclusion: These features help developers write organized, reusable, flexible, and maintainable programs.

Q2. What are Method Overloading and Method Overriding? Explain with examples.

Method overloading and method overriding are forms of polymorphism in Java. They allow methods to be used flexibly, but they work in different ways.

1. Method Overloading

Method overloading occurs when two or more methods in the same class have the same name but different parameter lists. The parameters can differ in number, type, or order. Changing only the return type is not enough to overload a method.

Example:

```
class Calculator {     int
add(int a, int b) {
return a + b;     }

    int add(int a, int b, int c) {
return a + b + c;     }

    public static void main(String[] args) {
Calculator c = new Calculator();
        System.out.println(c.add(2, 3));
        System.out.println(c.add(2, 3, 4));
    }
}
```

Output:

```
5
9
```

Explanation: Both methods are named **add()**, but one accepts two arguments and the other accepts three. The compiler selects the appropriate method based on the arguments.

2. Method Overriding

Method overriding occurs when a subclass provides its own implementation of an inherited instance method with the same signature. It allows a subclass to specialize the behaviour of its parent class.

Example:

```
class Animal {     void sound() {
System.out.println("Animal sound");
    }
}

class Dog extends Animal {     @Override
void sound() {
System.out.println("Dog barks");     }

    public static void main(String[] args) {
Dog d = new Dog();
        d.sound();
}
}
```

Output:

```
Dog barks
```

Explanation: The **Dog** class overrides the **sound()** method and provides its own behaviour. The **@Override** annotation helps the compiler check that the method is correctly overridden.

Differences:

- **Overloading:** Same method name, different parameter lists; commonly occurs in the same class.
- **Overriding:** Same method signature in a parent and subclass; requires inheritance.
- **Overloading** is compile-time polymorphism, while **overriding** is runtime polymorphism.

Q3. Write short notes on the following keywords.

a) Static Keyword

The **static** keyword declares a member that belongs to the class itself rather than to each individual object. Static variables are shared by all objects of the class, and static methods can be called using the class name.

```
class Student {     static String
college = "SKIT";

    public static void main(String[] args) {
System.out.println(Student.college);
    }
}
```

Output: **SKIT**

Here, **college** is shared by the class. No object is required to access it. A static method cannot directly access non-static instance members without an object.

b) Final Keyword

The **final** keyword is used to restrict changes. A final variable cannot be reassigned, a final method cannot be overridden, and a final class cannot be extended. `final int MAX = 100;`

The variable **MAX** cannot be assigned another value after initialization. The **final** keyword is useful when a value or behaviour must remain fixed.

c) Extends Keyword

The **extends** keyword establishes inheritance between classes. A subclass can use accessible members of its superclass and can add its own members or override inherited methods.

```
class Animal {     void eat() {
System.out.println("Eating");
    }
}

class Dog extends Animal {     void
bark() {
System.out.println("Barking");
    }
}
```

The **Dog** class inherits the **eat()** method from **Animal** and also defines its own **bark()** method. Java classes can directly extend only one class.

d) Implements Keyword

The **implements** keyword is used when a class agrees to implement an interface. The class provides implementations for the interface's abstract methods, unless the class itself is declared abstract. This supports abstraction and flexible program design.

```
interface Printable {
void print(); }

class Demo implements Printable {
public void print() {
System.out.println("Printing");
    }
}
```

Here, **Demo** implements the **Printable** interface and defines its **print()** method. Interface methods that are implicitly public must be implemented with public visibility.

e) Super Keyword

The **super** keyword refers to the immediate parent class. It is used to access a parent class variable, call a parent class method, or invoke a parent class constructor.

```
class Animal {     void sound() {
System.out.println("Animal sound");
    }
}

class Dog extends Animal {     void
sound() {         super.sound();
System.out.println("Dog barks");
    }
}
```

Output when **sound()** is called on a **Dog** object:

```
Animal sound
Dog barks
```

The statement **super.sound()** calls the parent class method before the subclass prints its own message. A call to **super()** in a constructor must be the first statement in that constructor.

PAPER 2

Q1. Write short notes on the following.

a) Interface

An interface in Java defines a contract that a class can implement. It can declare abstract methods and constants, and it can also contain default and static methods. A class uses the **implements** keyword to implement an interface.

```
interface Shape {
void draw(); }

class Circle implements Shape {     public
void draw() {
System.out.println("Drawing Circle");
    }
}
```

The **Circle** class provides the implementation of **draw()**. Interfaces support abstraction and allow a class to implement multiple interfaces.

b) Constructor

A constructor is a special member used to initialize an object when it is created. It has the same name as the class and has no return type, not even **void**.

Common types:

- **No-argument constructor:** Takes no parameters.
- **Parameterized constructor:** Accepts parameters to initialize an object with specific values.

Example:

```
class Student {
int rollNo;

    Student(int r) {
rollNo = r;     }
}
```

When **new Student(1)** is used, the constructor assigns **1** to **rollNo**. If no constructor is declared, Java provides a default constructor automatically.

c) Super Keyword

The **super** keyword refers to the immediate parent class. It can access parent class fields and methods and can call a parent constructor using **super()**.

```
class Parent {
int x = 10; }

class Child extends Parent {
int x = 20;

    void display() {
System.out.println(super.x);
        System.out.println(x);
    }
}
```

Output when **display()** is called:

```
10
20
```

Here, **super.x** accesses the parent's variable, while **x** accesses the child's variable.

d) Method Overriding

Method overriding occurs when a subclass provides its own implementation of an inherited instance method with the same signature. It is used to achieve runtime polymorphism.

```
class Animal {     void sound() {
System.out.println("Animal sound");
    }
}

class Dog extends Animal {     @Override
void sound() {
System.out.println("Dog barks");
    }
}
```

When **sound()** is called on a **Dog** object, the overridden method prints **Dog barks**. Overriding allows subclasses to provide behaviour appropriate to their type.

e) Method Overloading

Method overloading means defining multiple methods with the same name but different parameter lists. The compiler selects a method based on the arguments supplied.

```
class Demo {     void add(int a, int
b) { }     void add(int a, int b, int
c) { } }
```

The two **add()** methods are overloaded because they accept different numbers of parameters. Overloading is an example of compile-time polymorphism.

Q2. What is a package? How do you compile it? Explain with an example.

A package is a group of related classes and interfaces. Packages organize code, help prevent naming conflicts, and support access control.

Types of packages:

- **Built-in packages:** Provided by Java, such as **java.util** and **java.io**.
- **User-defined packages:** Created by programmers to organize their own classes.

Step 1: Create a file named Hello.java.

```
package mypack;

public class Hello {     public void
display() {
System.out.println("Hello Java");
    }
}
```

The **package mypack;** statement declares that the class belongs to the **mypack** package.

Step 2: Compile the package class.

```
javac -d . Hello.java
```

The **-d .** option tells the compiler to place the compiled class in the correct package directory under the current directory. This creates **mypack/Hello.class**.

Step 3: Create a file named Test.java.

```
import mypack.Hello;

class Test {
    public static void main(String[] args) {
Hello h = new Hello();
        h.display();
}
}
```

The **import** statement allows the program to refer to the **Hello** class by its simple name.

Step 4: Compile the test program.

```
javac -cp . Test.java
```

Step 5: Run the program.

`java -cp . Test` **Output:**

```
Hello Java
```

In these commands, **-cp .** sets the classpath to the current directory. Run the commands from the directory containing **Hello.java** and **Test.java**.

Advantages of packages:

- Organize related classes and interfaces.
- Reduce naming conflicts.
- Support access control.
- Make large programs easier to maintain and reuse.

Q3. What is the use of the final keyword? Explain with examples.

The **final** keyword in Java restricts changes. Its effect depends on whether it is applied to a variable, method, or class.

1. Final Variable

A final variable can be assigned only once. After it has been initialized, its value cannot be reassigned.

```
class Demo {     public static void
main(String[] args) {         final int MAX
= 100;         System.out.println(MAX);
    }
}
```

Output: **100**

If we try to assign another value to **MAX**, the compiler reports an error.

2. Final Method

A final method cannot be overridden by a subclass. It is useful when the implementation of a method must remain unchanged in subclasses.

```
class Parent {     final void display() {
System.out.println("Parent method");
    }
}

class Child extends Parent {     //
display() cannot be overridden here.
}
```

3. Final Class

A final class cannot be extended by another class. This prevents inheritance from that class.

```
final class Demo {     void show()
{
System.out.println("Java");
    }
}
```

A class declaration such as **class Child extends Demo** would cause a compilation error because **Demo** is final.

Conclusion: The **final** keyword helps protect values and prevent unwanted method overriding or class inheritance. Note that a final reference variable cannot point to a different object, but the object's internal state may still be changed if its class allows it.